Coding & Solution

This document presents the core implementation of OptiCrop, covering key code modules, ML training pipeline, Flask routing, and template rendering.

1. Dataset Overview

Source	Kaggle Crop Recommendation Dataset
Rows	2,200 samples (100 per crop class)
Columns	N, P, K, temperature, humidity, ph, rainfall, label (target)
Crop classes	22 (rice, maize, chickpea, kidneybeans, pigeonpeas, mothbeans, mungbean, blackgram, lentil, pomegranate, banana, mango, grapes, watermelon, muskmelon, apple, orange, papaya, coconut, cotton, jute, coffee)
Train/Test split	80/20 stratified 1,760 train | 440 test

2. Preprocessing Pipeline (app/ml/preprocess.py)

```
FEATURE_RANGES = {
    'N':           (0,    140),
'P':           (5,    145),
'K':           (5,    205),
    'temperature': (8.0,  44.0),
    'humidity':    (14.0, 100.0),
'ph':          (3.5,  9.5),
    'rainfall':    (20.0,
300.0), }

def validate_inputs(form_data: dict) -> dict:
validated = {}
    for key, (lo, hi) in FEATURE_RANGES.items():
val = float(form_data[key])
        if not lo <= val <= hi:
            raise ValueError(f'{key} out of range [{lo}, {hi}]')
validated[key] = val
    return validated
```

3. Prediction Engine (app/ml/predictor.py)

```
MODEL_PATH  = 'models/best_model.pkl'
SCALER_PATH = 'models/scaler.pkl'

def predict_crop(inputs: dict) -> tuple[str, float]:
model  = pickle.load(open(MODEL_PATH,  'rb'))
    scaler = pickle.load(open(SCALER_PATH, 'rb'))
    arr    = np.array([[inputs[k] for k in FEATURE_NAMES]])
scaled = scaler.transform(arr)
    label  = model.predict(scaled)[0]
```

```
    proba  = model.predict_proba(scaled).max()
    return label, round(float(proba) * 100, 2)
```

4. Flask Application (app.py)

```
@app.route('/', methods=['GET'])
def index():
    return render_template('index.html')

@app.route('/predict', methods=['POST'])
def predict():
    try:
        inputs = validate_inputs(request.form)
        crop, confidence = predict_crop(inputs)
return render_template('result.html',
                               crop=crop,
                               confidence=confidence,
inputs=inputs)
    except (ValueError, KeyError) as e:
flash(str(e), 'error')
        return redirect(url_for('index'))
```

5. Request Response Flow

Figure 6: HTTP Request Response Flow

6. Model Training Summary

Parameter	Value
Algorithm	RandomForestClassifier(n_estimators=100, random_state=42)
Train Accuracy	100.0% (no overfitting observed on validation set)
Test Accuracy	99.09% (437/440 correct on hold-out set)
Cross-Val (5-fold)	99.0% – 0.4% confirms generalisation
Feature Importance (top)	rainfall (0.22) > humidity (0.19) > K (0.17) > N (0.16) > P (0.14) > ph (0.07) > temp (0.05)